

QA 3 (Unit 3): Naive Bayes Implementation

Manual Calculation + Code Implementation

Date: June 15, 2026

1. Introduction

Naive Bayes is a probabilistic machine learning algorithm based on Bayes' Theorem. It assumes that all features are independent of each other given the class label — hence the term 'naive'. Despite this simplifying assumption, it performs surprisingly well on text classification tasks such as spam detection and sentiment analysis.

2. How Naive Bayes Works

Bayes' Theorem states:

$$P(\text{Class} \mid \text{Features}) \propto P(\text{Class}) \times P(\text{Features} \mid \text{Class})$$

- Prior $P(\text{Class})$: Probability of each class before seeing any features
- Likelihood $P(\text{Feature} \mid \text{Class})$: Probability of a feature given the class
- Posterior $P(\text{Class} \mid \text{Features})$: Updated probability after seeing features
- Prediction: Choose the class with the highest posterior

3. Dataset Description

A small spam detection dataset with 8 messages and binary labels (spam/ham).

Message	Words	Label
msg1	free win prize money	spam
msg2	free offer click now	spam
msg3	win free cash today	spam
msg4	hello how are you	ham
msg5	meeting tomorrow morning	ham
msg6	see you later today	ham
msg7	free cash win now click	spam
msg8	lunch tomorrow with team	ham

4. Part 1: Manual Naive Bayes Calculation

4.1 Test Message: ['free'=Yes, 'win'=Yes, 'hello'=No]

Step 1: Prior Probabilities

Class	Count	Prior
Spam	4	$P(\text{Spam}) = 4/8 = 0.5$
Ham	4	$P(\text{Ham}) = 4/8 = 0.5$

Step 2: Likelihoods (with Laplace Smoothing)

Feature	$P(\text{feat} \text{Spam})$	$P(\text{feat} \text{Ham})$	Formula
free=Yes	$3/5 = 0.60$	$1/5 = 0.20$	$(\text{count}+1)/(\text{total}+\text{vocab})$
win=Yes	$3/5 = 0.60$	$1/5 = 0.20$	$(\text{count}+1)/(\text{total}+\text{vocab})$
hello=No	$4/5 = 0.80$	$2/5 = 0.40$	$(\text{count}+1)/(\text{total}+\text{vocab})$

Step 3: Posterior Scores

Class	Score (unnormalized)
Spam	$0.5 \times 0.60 \times 0.60 \times 0.80 = 0.144$
Ham	$0.5 \times 0.20 \times 0.20 \times 0.40 = 0.008$

Prediction → SPAM (0.144 > 0.008)

5. Part 2: Code Implementation Steps

- Import libraries: numpy, pandas, sklearn.naive_bayes.MultinomialNB, CountVectorizer
- Create dataset: 8 messages with spam/ham labels
- Vectorize: Convert text to word count matrix using CountVectorizer
- Train: Fit MultinomialNB on training data (70/30 split)
- Predict: Get class predictions and probabilities
- Manual NB: Implement from scratch using log probabilities
- Compare: Manual vs Sklearn results side by side

6. Expected Output

Metric	Manual NB	Sklearn NB
Accuracy	~100%	~100%
Spam Detection	✓ Correct	✓ Correct
Ham Detection	✓ Correct	✓ Correct

Predictions Match	Yes	Yes
-------------------	-----	-----

7. Manual vs Sklearn Comparison

Feature	Manual NB	Sklearn NB
Implementation	From scratch	Built-in library
Smoothing	Laplace (+1)	Laplace (alpha=1)
Log probabilities	Yes	Yes
Speed	Slower	Faster
Accuracy	Same	Same

8. Conclusion

Both the manual implementation and sklearn's MultinomialNB produced identical predictions, confirming that Naive Bayes is easy to implement from scratch. The algorithm works well for text classification because:

- It is fast to train and predict
- It works well even with small datasets
- The independence assumption, while naive, performs well in practice

9. References

- Scikit-learn Documentation: https://scikit-learn.org/stable/modules/naive_bayes.html
- Bayes, T. (1763). An Essay towards solving a Problem in the Doctrine of Chances.